Ficha da Unidade Curricular 2025/2026

IDENTIFICAÇÃO DA UNIDADE CURRICULAR	
Designação:	Programação Orientada a Objetos
Última alteração:	2026-02-16 17:40:00
ECTS:	6
Língua de lecionação:	Português
Cursos onde é lecionada:	Licenciatura em Engenharia Informática 1º Ano, 2º Semestre

CORPO DOCENTE

Prof. Doutora Maria Paula Coutinho Dias Morais (Regente)

Prof. Doutor Tiago Silva Oliveira Bastos (Docente)

OBJETIVOS DA APRENDIZAGEM

1. Identificar as características dos paradigmas e linguagens de programação.
2. Implementar composição entre classes
3: Implementar herança entre classes
4. Implementar aplicações orientadas a objetos usando Java.
5. Otimizar aplicações OO usando Java.

O ODS 4 enquadra-se nos objetivos desta UC, uma vez que os métodos de ensino e avaliação usados contribuirão para que os estudantes sejam capazes de reconhecer a importância das próprias habilidades para melhorar a sua vida e também serão mais capazes para motivar colegas para aproveitarem oportunidades educacionais.

PRÉ-REQUISITOS RECOMENDADOS

Conhecimentos adquiridos nas unidades curriculares:
Algoritmia e Programação
Álgebra Linear e Geometria Analítica: Raciocínio lógico de demonstração e operações sobre matrizes
Competências Comportamentais:planear, gerir e desenvolver um trabalho de grupo, bem como apresentá-lo, quer escrita, quer oralmente.

MÉTODO DE ENSINO

As aulas têm tipologia téorico/prática (TP) e prática laboratorial (PL). Privilegiar-se-ão estratégias de aprendizagem ativa para fomentar um maior empenho dos

estudantes: sala de aula invertida e think-pair-share nas aulas TP; em algumas PL utilizar-se-á também peer-review.
Utilizar-se-á aprendizagem baseada em projeto, com o objetivo de desenvolvimento de um projeto interdisciplinar, usando uma abordagem Scrum.
Em algumas aulas teórico-práticas utilizar-se-á, também, o método expositivo para apresentação de conceitos, recorrendo, frequentemente, à interpelação aos alunos e ao ensino pela descoberta através de exemplos.
As aulas práticas versarão, também, a resolução de exercícios práticos no computador, com acompanhamento individual pelo professor.
O projeto interdisciplinar incluirá 2 UCs do 1º ano da licenciatura em EI (SI, POO), tendo como objetivos a integração de diferentes conhecimentos, uma melhor compreensão da interligação das temáticas abordadas nas 2 UCs e estimular o desenvolvimento de competências soft: o trabalho em equipa, resolução de problemas e o pensamento crítico. No âmbito do projeto os estudantes farão uma auto-avaliação e também uma avaliação dos pares.

CONTEÚDOS PROGRAMÁTICOS

1. Linguagens e paradigmas de programação
1.1. Paradigmas: imperativo, orientado a objetos (POO), funcional, lógico e declarativo
1.2. Exemplos de linguagens de programação
2. Programação Orientada a Objetos
2.1. Classes e objetos
2.2. Construtores, métodos, parâmetros
2.3. Polimorfismo por overloading
2.4. Encapsulamento - uso de modificadores de acesso (public, private)
2.5. Boas práticas na escrita de classes
3. Relações entre classes - Composição
3.1. Relação “tem-um” (has-a relationship)
3.2. Uso de listas para representar objetos compostos
4. Relações entre classes - Herança
4.1. Relação “é-um” (is-a relationship)
4.2. Benefícios da reutilização de código
4.3. Hierarquia de classes
4.4. Classes abstratas e interfaces
4.5. Tipos estático e dinâmico
4.6. Polimorfismo por overriding
4.7. Procura dinâmica de métodos
5. Java
5.1. JDK,JRE, máquina virtual
5.2. Tipos primitivos e Strings
5.3. Arrays
5.4. Estruturas de controlo e repetição
5.5. Estrutura base de um programa Java
5.6. Definição de classes e criação de objetos
5.7. Wrapper classes
5.8. I/O Simples
5.9. A Java Collections Framework
6. IDEs e outras ferramentas de suporte ao desenvolvimento de software
6.1 Objetivos e vantagens de um IDE
6.2 Utilização do BlueJ e do Eclipse

6.3 Ferramentas de Debugging
7. Qualidade no desenho de software
7.1 Boas práticas na escrita de programas em Java
7.2 Desenho orientado a responsabilidades, coesão, acoplamento, encapsulamento
8. Testes e Debugging
8.1 Conceito e importância do debugging no desenvolvimento de software
8.2 Técnicas de Debugging
8.3 Análise e tratamento de exceções

BIBLIOGRAFIA RECOMENDADA

Schildt, H., & Coward, D. (2024). Java: The complete reference (13ª ed.). McGraw Hill.
Liang, D. (2020). Introduction to Java programming and data structures: Comprehensive version (12ª ed.). Pearson.
Barnes, D., & Kolling, M. (2017). Objects first with Java (6ª ed.). Prentice-Hall/Pearson Education.
Martins, M. (2017). Java 8: POO + Construções Funcionais. FCA.

BIBLIOGRAFIA COMPLEMENTAR

Java Documentation:https://docs.oracle.com/en/java/index.html
The Java™ Tutorials: http://docs.oracle.com/javase/tutorial/index.html
Trail: Learning the Java Language: https://docs.oracle.com/javase%2Ftutorial%2F/java/index.html
Rubin, K. (2013). Essential Scrum: a practical guide to the most popular agile process. Addison-Wesley

PLANIFICAÇÃO SEMANAL

Aulas Teórico-Práticas

1.Linguagens e Paradigmas de programação: breve apresentação de diferentes Paradigms/linguagens de programação. Introdução à linguagem Java. Componentes do Java SE.O processo de escrita e execução de um programa em Java. A Java Virtual Machine. Vantagem do uso de Ambientes integrados de desenvolvimento (IDEs) Escrita de um 1º programa em Java - diferenças para Python.

2.A Linguagem Java: tipos primitivos; vetores e matrizes; Estruturas de controlo e repetição. Exemplos. Utilização de métodos - vantagens; parâmetros formais e reais; âmbito e visibilidade das variáveis; métodos e variáveis static.; passagem de parâmetros por valor – exemplos

3.O paradigma da POO: conceitos de classes e objetos; variáveis e métodos de classe e de instância; construtores; polimorfismo por sobrecarga (overloading) - utilização de metodos de aprendizagem ativa para compreensão destes conceitos

4.Visibilidade/Âmbito e tempo de vida de uma variável em java. O conceito de Java Pacotes/Packages e sua utilização. Modificadores de visibilidade: public e private. Conceito de abstração e encapsulamento; utilização de métodos interrogadores/modificadores no desenho de uma classe. O método toString.

5.Resolução de problemas para compreensão e aplicação dos conhecimentos sobre classes e objetos; diagramas de classes e diagramas de objetos. Qualidade no desenho de classes: acoplamento e coesão; desenho orientado por responsabilidades; boas práticas na escrita de classes; Visibilidade das váriáveis - o encapsulamento.

6.Relações entre classes: associação - diferença entre composição e agregação; resolução de casos práticos para identificação das classes, atributos e relações e compreender como desenhar a arquitetura de um projeto, desenho dos respetivos diagramas de classes. Boas práticas na escrita de classes: desenho orientado por responsabilidades, coesão, encapsulamento.

7.Discussão de um exemplo prático (Turmas/Alunos) para implementação, em Java, de uma aplicação muito simples de gestao de notas de alunos usando diferentes classes. Compreender, atraves do exemplo os conceitos de abstração e encapsulamento.

8.O IDE Eclipse. Boas práticas na organização de um projeto. Utilização de coleções do Java: ArrayList, HashSet e HashMap; iteradores.

9.Relações entre classes. Is a/Herança: compreensão dos principais conceitos através de um exemplo prático. Polimorfismo por sobreposição; Tipos estático e dinâmico. Procura dinâmica de métodos.. Resolução guiada de um exercício para discussão dos conceitos de herança.

10.Herança. Classes abstratas, métodos abstratos - exemplos práticos.

11.Implementação de herança múltipla em Java. Utilização de classes Interface. - exemplos práticos.

12.A utilização de Sistemas de Controlo de versões de software - conceitos principais sobre Git e GitHub. Esclarecimento de dúvidas sobre o projeto.

13.Tratamento de exceções em java - principais conceitos e exemplos concretos

14.Esclarecimento de dúvidas / acompanhamento dos projetos

15.Esclarecimento de dúvidas / acompanhamento dos projetos

16.Discussão do código do 2º incremento do projeto

Praticas laboratoriais

1.Apresentação da UC: objetivos de aprendizagem, metodologia de ensino e avaliação, bibliografia, esforço de trabalho (ECTS). Revisões de algoritmia

2.Revisões Algoritmia

3.Paralelismo algoritmia/python/java; Utilização do BLueJ - a escrita de programas em Java. Resolução de exercícios práticos para compreensão do funcionamento de arrays e matrizes em java
4.Resolução de exercícios práticos para compreensão do conceito de método e respetiva implementação em Java
5.Resolução de exercícios práticos para compreensão do conceito de método e respetiva implementação em Java; Resolução de exercícios para manipulação de Strings em java.
6.Resolução de problemas para compreensão e aplicação dos conhecimentos sobre classes e objetos; Utilização do JavaDoc para criação de documentação profissional em Java.
7.Relações entre classes: Has a/ Composição. Resolução de exercícios para Implementação de composição entre classes.
8.Relações entre classes: Has a/ Composição. Resolução de exercícios para Implementação de composição entre classes. Utilização do package java.time para manipulação de datas/tempos. Utilização de ferramentas de ebugging
9.Implementação de aplicações orientadas a objetos usando Eclipse. Utilização da classe ArrayList para gestão de listas. Exemplos do uso de Wrapper classes do Java.
Trabalho conjunto com a unidade curricular Sistemas de Informação: implementação de User stories
10.Resolução de exercícios para Implementação de herança entre classes.
Trabalho conjunto com a unidade curricular Sistemas de Informação: implementação de User stories
11.Resolução de exercícios para Implementação de herança entre classes.
12.Projeto interdisciplinar - 1º Sprint planning.
Implementação de aplicações orientadas a objetos usando Java.
13.Desenho da arquitetura do projeto; implementação do 1º incremento
14.1º sprint review/retrospective; 2º sprint plannning Discussão do código do 1º incremento
Esclarecimento de dúvidas / acompanhamento dos projetos- 2º sprint
15.Esclarecimento de dúvidas / acompanhamento dos projetos
16.2º sprint review/retrospective

DEMONSTRAÇÃO DA COERÊNCIA DOS CONTEÚDOS PROGRAMÁTICOS COM OS OBJETIVOS

No tópico 1 discutem-se os paradigmas e linguagens de programação, contribuindo para identificar o objetivo 1. No tópico 2 são apresentados os conceitos fundamentais e construções intrínsecas ao paradigma OO contribuindo para atingir os objetivos 2, 3 e 4. No tópico 3 discute-se o conceito de composição, contribuindo para o objetivo 2 e 4. No tópico 4 discutem-se os conceitos de herança e abstração e modularização contribuindo para que o estudante resolva problemas usando herança entre classes (objetivos 4 e 5). No tópico 5 são apresentados os conceitos fundamentais da linguagem Java que habilitarão o estudante a implementar aplicações OO em Java. O tópico 6 permite ao estudante conhecer e usar ambientes integrados de desenvolvimento, contribuindo para o obj. 4. Os tópicos 7 e 8 contribuem para os obj 4 e 5.

DEMONSTRAÇÃO DA COERÊNCIA DAS METODOLOGIAS DE ENSINO COM OS OBJETIVOS

As estratégias de sala de aula invertida, think-pair-share e o método expositivo usado nas aulas teórico-práticas permitem a compreensão dos conceitos e construções intrínsecas ao paradigma de programação orientada a objetos. As aulas laboratoriais destinam-se a que os alunos resolvam problemas e realizem projetos onde aplicam esses conhecimentos, apercebendo-se das dificuldades práticas da sua aplicação e cimentando os conceitos teóricos, permitindo-lhes que

sejam capazes de criar aplicações orientadas a objetos usando Java. Nestas aulas utilizar-se-ão, frequentemente, as estratégias de problema e projeto, adotando uma abordagem SCRUM, para melhor compreensão da aplicação dos conceitos e desenvolvimento de competencias transversais como gestão de tempo, trabalho em equipa e comunicação escrita e oral. Usar-se-á também a estratégia de peer-review para desenvolvimento do espírito critico dos estudantes.

DISTRIBUIÇÃO DAS HORAS DE CONTACTO POR SEMESTRE	
Ensino Teórico-Prático [TP]	30
Ensino Prático ou Laboratorial [PL]	60
Total:	**90**

DISTRIBUIÇÃO DAS HORAS DE TRABALHO INDEPENDENTE POR SEMESTRE	
Projecto de grupo	32
Provas (escritas, orais, práticas, etc.)	8
Investigação (recolha e análise de dados, etc.)	4
Estudo autónomo	28
Total:	**72**

COMPETÊNCIAS GENÉRICAS RELEVANTES	DESENVOLVIDA	AVALIADA
Análise e síntese	✓	✓
Aptidão para aplicação na prática dos conhecimentos teóricos	✓	✓
Capacidade crítica e de avaliação	✓	✓
Capacidade de adaptação a novas situações	✓	—
Capacidade de auto-critica e de auto-avaliação	✓	✓
Capacidade de decisão	—	—
Capacidade de investigação	✓	—
Competência em informática e uso de novas tecnologias	✓	✓
Competência em língua estrangeira	✓	—
Comportamento ético e responsável	✓	✓
Comunicação oral e escrita	✓	✓
Criatividade	✓	✓
Preocupação com a eficácia	✓	✓
Preocupação com a qualidade	✓	✓
Relacionamento interpessoal	✓	✓
Resolução de problemas	✓	✓
Saber organizar, planear e gerir	✓	✓

Trabalho em equipa	✓	✓

OBJETIVOS DE DESENVOLVIMENTO SUSTENTÁVEL
4. Garantir o acesso à educação inclusiva, de qualidade e equitativa, e promover oportunidades de aprendizagem ao longo da vida para todos

PERMISSÃO DE UTILIZAÇÃO DE IA GENERATIVA
Permitida a utilização de IA generativa: Sim
Para que efeitos: Como auxiliar de estudo. Como assistente na escrita de programas, na perspetiva de "tutor" e com o objetivo dos estudantes perceberem as vantagens e limitações do uso de diferentes ferramentas. Não permitida em momentos de avaliação.
Ferramentas autorizadas: Qualquer ferramenta de GenAI
Forma de declaração de utilização: Não se aplica, uma vez que não é permitido o seu uso em momentos de avaliação.

AVALIAÇÃO DE CONHECIMENTOS E COMPETÊNCIAS	
Método de avaliação:	Contínua.
Observações:	

ÉPOCA NORMAL	PESO	NOTA MÍNIMA	CONDICIONADO?
Assiduidade	—	70% presenças	—
Participação/Substituição - Empenho e envolvimento nas aulas	25%	10,0 valores	—
Prova Escrita – Fichas e mini-testes - Quiz de avaliação formativa e sumativa	10%	—	—
Prova Escrita – Fichas e mini-testes - Quiz de avaliação formativa e sumativa	10%	—	—
Prova prática - Prova prática individual	25%	9,0 valores	—
Trabalho - Projeto interdisciplinar de grupo com apresentação	30%	10,0 valores	—
Observações:	Será considerada a assiduidade. Para que possam ser avaliados na componente de Participação, os alunos com estatuto ou, que por qualquer outra razão justificada e presente no Regulamento pedagógico, não possam participar nas aulas, deverão fazer as atividades previstas no MOODLE e entregar os "resultados" no MOODLE, nas datas indicadas e nas pastas devidamente identificadas. As componentes da avaliação da Participação estão descritas no MOODLE.		

ÉPOCA RECURSO	PESO	NOTA MÍNIMA	CONDICIONADO?
Prova Escrita – Exame Final/ Equivalente - Prova escrita	35%	8,0 valores	—
Prova prática - Prova prática individual	35%	9,0 valores	—
Trabalho - Projeto interdisciplinar de grupo com apresentação	30%	10,0 valores	—
Observações:	Os alunos que entregaram trabalho na época normal podem, se assim o desejarem, manter a nota obtida. Para melhoria da nota da época normal, o estudante terá sempre que fazer a prova escrita, podendo manter as notas das outras duas componentes.		

ÉPOCA ESPECIAL	PESO	NOTA MÍNIMA	CONDICIONADO?
Prova Escrita – Exame Final/ Equivalente - Prova escrita	50%	8,0 valores	—
Prova prática - Prova prática individual	50%	9,0 valores	—
Observações:			